



Assignment 2

Aditya Fadhillah (hig708)

TA: Therese Lyngby

Program Analysis and Transformation

Department of Computer Science, May 5, 2025

Program Inversion

1. Janus Run-Length Encoder

After running the Janus `encode` procedure with the initial store `text[10] = {7, 7, 7, 3, 3, 5, 5, 5, 5, 0}`, the final state of the store is:

```
code[20] = {7, 3, 3, 2, 5, 4, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
i = 0
j = 0
text[10] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
```

The Janus encoder performs run-length encoding by scanning the array `text` and recording each repeated values into the `code` array. For each run, it stores a pair: the value itself and the number of times it occurs consecutively. During encoding, the values from the input array `text` are replaced by zeros, and the variables `i` and `j` are returned to zero at the end, making the procedure reversible.

2. Run-length encoder in a conventional imperative language

The following is the encoder written in C#:

```
static void Encode(int[] text, int[] code)
{
    int i = 0;
    int j = 0;

    while (text[i] != 0)
    {
        code[j] = text[i];

        while (text[i] == code[j])
        {
            code[j + 1]++;
            i++;
        }

        j += 2;
    }
}
```

```
-----
code[]:
7 3 3 2 5 4 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

The Janus version enforces reversibility, which makes sense because Janus is a reversible language. Each operation (`+=`, `<=>`) and control flow (`from-loop-until`) can be reversed, and no data is discarded. The loop counters `i` and `j` are also restored to zero at the end of execution to maintain reversibility.

In contrast, the C# version uses standard `while` loops. It does not preserve the input `text[]` or the intermediate values needed reconstruct it (`i` and `j`), making the program irreversible. Also, the use of mutable state and overwriting operations like `=` will inherently make the program lose information.

3. Invert the Janus encoder

```
i j text[10] code[20]
```

```
procedure decode
  from i = 0 && j = 0 do
    i += code[j+1]
    j += 2
  until code[j] = 0
  from text[i] = 0 loop
    j -= 2
    from code[j] != text[i] do
      i -= 1
      code[j+1] -= 1
    loop text[i] += code[j]
    until code[j + 1] = 0
    code[j] <=> text[i]
  until i = 0 && j = 0
```

```
procedure main
  code[0] += 7
  code[1] += 3
  code[2] += 3
  code[3] += 2
  code[4] += 5
  code[5] += 4
  code[6] += 0
  call decode
```

```
-----
code[20] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0}
i = 0
j = 0
text[10] = {7, 7, 7, 3, 3, 5, 5, 5, 5, 0}
```

4. Inversion rules used for the inversion of encode

The inversion of the Janus encoder into the decoder is based on the following general program inversion rules:

- The statement `a += b` is inverted using `a -= b`
- The swap operation `a <=> b` is its own inverse
- Loop structures of the form **from-loop-until** are inverted by reversing their direction:
The forward loop accumulates (`i += 1`), while the reverse loop subtracts (`i -= 1`).
The inversion also restores the values in `text[]` using the values and counts stored in `code[]`.
- The symmetry of the condition that resets both `i` and `j` to zero ensures that the program remains reversible.

5. Invert the encoder in a conventional imperative language

An attempt has been made to show the difficulties of inverting a C-like encoder, which resulted in a decoder that runs the encoding logic backward, similar to how Janus supports reversal:

```
static void Decode(int[] text, int[] code)
{
    int i = 0;
    int j = 0;

    while (text[i] != 0)
    {
        j -= 2;

        while (text[i] == code[j])
        {
            i--;
            code[j + 1]--;
        }

        code[j] = text[i];
    }
}
```

This approach mirrors the reverse logic of the Janus `encode` procedure but fails in `C#` because the language is not designed to preserve reversibility. Variables like `i` and `j` cannot be rewound. The encoder discards intermediate state, making it impossible to recover step counters or overwritten values. Control flow in `C#` is also directional and lacks the symmetric structure of `from-loop-until` used in Janus.

Reversible discrete simulation

6. Simulation for free-falling objects in Janus

Task 6: Discrete Free-Fall Simulation in Janus

a. Janus program

```
t v h tend

procedure fall
  from t = 0 loop
    t += 1
    v += 10
    h -= v
    h += 5
  until t = tend

procedure main
  t += 0
  v += 0
  h += 176
  tend += 3
  call fall
```

b. Manual execution trace (forward run)

Initial store:

`t = 0, v = 0, h = 176, tend = 3`

Statement	t	v	h	tend
from t = 0 (true)	0	0	176	3
t += 1	1	0	176	3
v += 10	1	10	176	3
h -= v	1	10	166	3
h += 5	1	10	171	3
until t = tend (false)	1	10	171	3
t += 1	2	10	171	3
v += 10	2	20	171	3
h -= v	2	20	151	3
h += 5	2	20	156	3
until t = tend (false)	2	20	156	3
t += 1	3	20	156	3
v += 10	3	30	156	3
h -= v	3	30	126	3
h += 5	3	30	131	3
until t = tend (true)	3	30	131	3

Final store:

t = 3, v = 30, h = 131, tend = 3

At each time step, the simulation increases time by 1 second and velocity by 10 m/s, and updating height based on the current velocity and a correction factor.. After 3 seconds, the object have a velocity of 30 m/s and a height of 131 meters.

0.1 Solve the inverse problem of a falling object

The object is dropped from the top of a tower and hits the ground after 4 seconds with the final velocity of 40 m/s. With Janus it is possible to calculate the height of the tower by running the program backward.

Running the following main after program fall, will allow Janus to run the program backward, without having to manually invert the fall program:

```

t += 4
v += 40
h += 0
tend += 4
uncall fall

```

After running the program in reverse, the resulting store is:

```

h = 80
t = 0
tend = 4
v = 0

```

Therefore, the object was dropped from a height of 80 meters.